

SECTION 7 COPROCESSOR INTERFACE

This section describes the coprocessor interface with respect to the communication protocol utilized by the MC68881/MC68882 (FPCP) and MC68020/MC68030 (MPU). This communication protocol includes electrical and command-level mechanisms that allow a coprocessor to act as an extension to the main processor.

The connection between the MPU and the FPCP is an extension of the M68000 bus interface, with the FPCP connected as an auxiliary device to the MPU. The FPCP is selected by a chip-select ($\overline{\text{CS}}$) signal that is decoded from the MPU function code and address bus lines.

The FPCP contains a set of coprocessor interface registers (CIRs) by which the main processor and coprocessor communicate. These registers are not related to the programming model implemented by the FPCP. Rather, they are used as communication ports that have specific functions associated with each register. When the FPCP is used as a coprocessor to the MPU, the programmer is never required to explicitly access these interface registers, since the coprocessor interface is implemented in the hardware and microcode of the MPU. A main processor other than an MPU explicitly accesses the FPCP CIRs using a software routine that simulates the behavior of the MPU with respect to the coprocessor interface.

For more information on the electrical interconnection between the main processor and the FPCP, refer to **SECTION 11 INTERFACING METHODS**.

7.1 CHIP-SELECT DECODE

The MPU does not require any special bus signals, beyond the normal M68000 Family bus control signals, for connection to the FPCP. The former MC68000 interrupt acknowledge address space (function code 111) is extended in the MPU to become the CPU address space. A portion of this space, identified by the MPU address bus, is dedicated to coprocessor devices. Figure 7-1 illustrates the information presented on the MPU address bus for coprocessor accesses in the CPU address space.

During CPU space cycles, address bits A19–A16 indicate the CPU space function that the main processor is performing. The MPU utilizes four of the possible 16 encodings of A19–A16 as listed in Table 7-1.

The coprocessor identification (Cp-ID), A15–A13, is taken from the coprocessor instruction operation word (refer to **7.4 COPROCESSOR INSTRUCTIONS**). The coprocessor interface

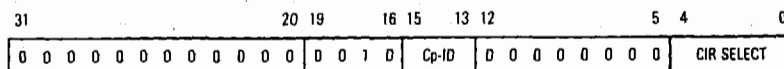
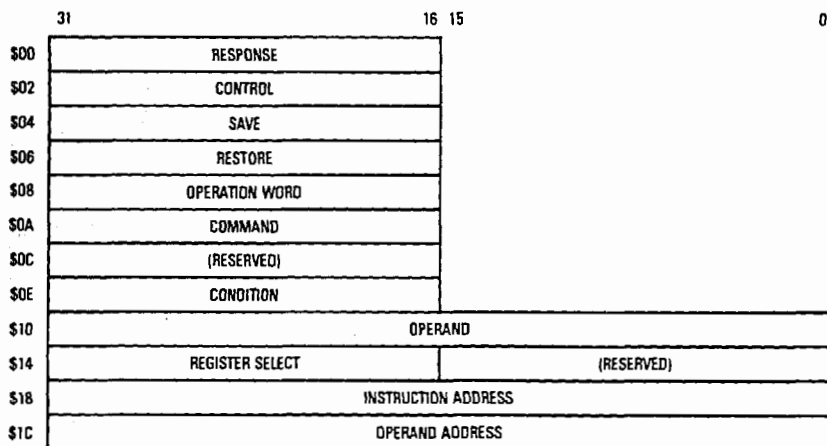


Figure 7-1. MPU Address Bus Encoding for Coprocessor Accesses

Table 7-1. MPU CPU Space Type Field Encoding

CPU Space Type Field (A19–A16)	CPU Space Transaction
0000	Breakpoint Acknowledge
0001	Access Level Control
0010	Coprocessor Communications
1111	Interrupt Acknowledge

register (CIR select) field, A4–A0, is decoded by the FPCP to select the appropriate CIR. For a map of the FPCP coprocessor interface registers in the CPU address space, refer to Figure 7-2. Since address bits A31–A20 are not present on all implementations of M68000 processors, these bits are not essential for decoding CPU space transactions and therefore are don't care bits.


Figure 7-2. FPCP Coprocessor Interface Register Map

The FPCP chip-select decode, therefore, uses the MPU function codes (FC2–FC0), the CPU space type field (A19–A16), and the Cp-ID field (A15–A13). The FPCP decodes the address bits A4–A0 to determine the function (as defined by the selected CIR) of any coprocessor access.

7.2 COPROCESSOR INTERFACE REGISTERS

Table 7-2 identifies the FPCP coprocessor interface register (CIR) locations in the CPU space that are used for communications between the MPU and the FPCP. Figure 7-2 illustrates the memory map of the CIRs on a 32-bit bus. When $\overline{CS}$ is asserted, the FPCP decodes the CIR select field of the address bus (A4–A0) to select the appropriate coprocessor interface register.

When the FPCP is used on a 32-bit bus, the coprocessor interface registers appear at the logical addresses shown in Figure 7-2 and Table 7-2. The M68000 dynamic bus sizing

col is used to place all word registers on the upper word of the data bus (D31–D16). s accomplished by asserting **DSACK1** and leaving **DSACK0** negated when any word register (other than the register select CIR) is accessed, regardless of the value of A1.

The following paragraphs describe the characteristics of each of the coprocessor interface registers as implemented by the FPCP. In each description, the read/write attribute of each register is included. If a register is read only, write accesses to that location are ignored; while read accesses of a write-only register always return all ones. In all cases, the FPCP asserts **DSACKx** in response to the assertion of **CS** in order to terminate the bus cycle.

Table 7-2. Coprocessor Interface Register Characteristics

Register	A4-A0	Offset	Width	Type
Response	0000x	\$00	16	Read
Control	0001x	\$02	16	Write
Save	0010x	\$04	16	Read
Restore	0011x	\$06	16	R/W
Operation Word*	0100x	\$08	16	R/W
Command	0101x	\$0A	16	Write
(Reserved)	0110x	\$0C	16	—
Condition	0111x	\$0E	16	Write
Operand	100xx	\$10	32	R/W
Register Select	1010x	\$14	16	Read
(Reserved)	1011x	\$16	16	—
Instruction Address	110xx	\$18	32	Write
Operand Address*	111xx	\$1C	32	R/W

*These CIRs are optionally implemented by a coprocessor only if they are needed; since they are not used by the MC68881, they are not implemented. Writes to these locations are ignored, and reads always return all ones.

7.2.1 Response CIR (\$00)

This 16-bit read-only register is used to communicate service requests from the FPCP to the main processor. A read of the response CIR is always legal, regardless of the state of an instruction dialog. The formats of the response primitives that are returned through this register are detailed in **7.4.2 Response Primitives**.

The execution of an instruction by the FPCP does not start until the main processor reads the response CIR for the first time after a write to the command CIR. Furthermore, a read of a primitive from the response CIR usually causes the FPCP to proceed to the next state in an instruction dialog. For example, if an evaluate effective address and transfer data primitive is encoded in the response CIR and the main processor reads that primitive, it is assumed that the primitive was successfully transferred (and saved for later use, if necessary) and that the requested service is performed. In this case, the FPCP then changes the encoding of the response CIR to the null primitive and waits for an access of the operand CIR to transfer the operand.

This 16-bit write-only register is utilized by a main processor to issue an exception acknowledge or instruction abort to the FPCP. Figure 7-3 illustrates the format of this register. Only two of the 16 bits are defined: the exception acknowledge (XA) and abort (AB) bits.

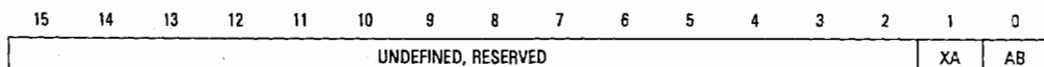


Figure 7-3. Control CIR Register

The implementation of the MC68881 does not utilize these two bits; instead, it interprets a write to this CIR address as an abort command, regardless of the data pattern written. Thus, an exception acknowledge (in response to a take exception primitive) or abort (in response to an illegal format word or an invalid primitive request) issued during any MC68881 instruction protocol, or an explicit write (e.g., with the MOVES instruction) to the control CIR always has the same effect on the MC68881. Also, write cycles to this register are never illegal since the MC68881 always responds in the same manner.

7

The response of the MC68881 to a write of the control CIR is:

1. To immediately terminate processing for any instruction that may be in progress. If an arithmetic instruction is in progress when an abort is issued, the content of the destination floating-point data register is undefined. No other user-visible registers are disturbed.
2. To clear any pending exceptions.
3. To reset the bus interface unit to the idle condition. Thus, the MC68881 is ready to begin a new instruction protocol following the write cycle.

Unlike the MC68881, the MC68882 distinguishes a write to the AB bit from a write to the XA bit. A write to the AB bit is interpreted as an abort of the last instruction received. However, an abort is only recognized during a certain window, which begins when the main processor writes an instruction to the command CIR and extends to the last CIR read or write required for that instruction. If the write to the AB bit of the control CIR occurs during this abort window, an abort function is initiated. Otherwise, a write to the AB bit is undefined and produces an undefined result. The response of the MC68882 to a write to the AB bit is:

1. To immediately terminate the instruction to which the abort window applies. Any concurrent instruction in progress within the MC68882 is allowed to complete.
2. To reset the bus interface unit to the idle condition, leaving the MC68882 ready to begin a new instruction protocol following the write cycle.

The write to the XA bit signals the MC68882 that the main processor is responding to an MC68882-detected exception. This write operation is necessary to clear any pending exceptions. However, the write operation alone does not guarantee that the exception is cleared. For the floating-point exception traps, it is the responsibility of the exception handler to clear the exception. If the handler does not clear the exception, the MC68882 continues reporting the same exception every time the main processor reads the response CIR. Refer to **5.2.2 Exception Handler Code** for additional exception handler requirements.

This 16-bit read-only register is used by the main processor to issue a context save command to the FPCP, and by the FPCP to return the format word of the FPCP state frame to the main processor. A read of this register causes any operation that may be executing (except a state save or restore) to be suspended, and a state save operation is initiated.

Following the read of a *not-ready, come-again* format word from the save CIR, the next expected access is a read of the save CIR. After the read of an idle or busy format word, the next expected access is to the operand CIR (to transfer the state frame). After the read of a null format word, the FPCP is in the reset state, and the next expected access is to the command or condition CIR.

The only time that a read of this register is illegal is when the FPCP is executing a state frame transfer for an FSAVE or FRESTORE instruction; a read of the save CIR is legal at any other time. If the main processor reads the save CIR at an illegal time, the invalid format word is returned. In response to the invalid format word, the main processor can write an abort to the FPCP to return it to the idle state.

7.2.4 Restore CIR (\$06)

This 16-bit read/write register is used by the main processor to issue a context restore command to the FPCP and to validate the format word of a state frame. A write of this register causes the FPCP to immediately stop any operation that may be executing and prepare to load a new internal state context from a memory-resident state frame.

After the main processor writes a format word to the restore CIR, it must read the restore CIR to receive the result of the format word verification. If the written format word is valid, that format word is read back from the restore CIR to indicate the successful verification. If the format word is invalid, the invalid format, take exception value is placed in the restore CIR to indicate the verification failure. After a successful verification is signaled, the next expected access is to the operand CIR (to transfer the state frame). After a verification failure is signaled, the main processor should write an abort to the control CIR in order to return the FPCP to the idle state. (The MPU does this automatically.)

7.2.5 Operation Word CIR (\$08)

This 16-bit write-only register is not used by the FPCP. The only time that this CIR location is used by the M68000 Family coprocessor interface is when a coprocessor issues the transfer operation word primitive, in which case the main processor writes the F-line word of the instruction to the operation word CIR. Since the FPCP never issues the transfer operation word primitive, the operation word CIR location should never be written by the main processor. If a write to this location occurs, it is ignored; it does not cause a protocol violation.

7.2.6 Command CIR (\$0A)

This 16-bit write-only register is used by the main processor to initiate the dialog for a general type coprocessor instruction. When the FPCP detects a write to this CIR location,



data value is latched from the data bus. If the MC68881 is executing a previous instruction, the APU or if the CU of the MC68882 is still busy when the command CIR is written, the latched command word is saved for later use, and the response CIR is encoded with the null (CA=1, IA=1) primitive. If the FPCP is in the idle or reset state when a write to the command CIR occurs, it encodes the first primitive for the selected instruction dialog in the response CIR in order to begin the execution of the new instruction.

A write to this CIR location is legal at any time except when the FPCP is in the initial phase of a general instruction or before the read of the conditional evaluation for a previous conditional instruction. If a write to the command CIR occurs when it is not expected, a protocol violation occurs, and the command word that is written is not saved by the FPCP.

7.2.7 Condition CIR (\$0E)

This 16-bit write-only register is used by the main processor to initiate the dialog for a conditional type coprocessor instruction. When the FPCP detects a write to this CIR location, the data value is latched from the data bus. If the FPCP is executing a previous instruction when the condition CIR is written, the latched conditional predicate is saved for later use, and the response CIR is encoded with the null (CA=1, IA=1) primitive. If the FPCP is in the idle or reset state when a write to the condition CIR occurs, it evaluates the selected condition and returns the null (CA=0, TF=x) primitive (where the TF bit indicates whether the conditional evaluation is true (1) or false (0)).

A write to this CIR location is legal at any time except when the FPCP is in the initial phase of a general instruction, or before the read of the conditional evaluation for a previous conditional instruction. If a write to the condition CIR occurs when it is not expected, a protocol violation occurs, and the conditional predicate that is written is not saved by the FPCP.

7.2.8 Operand CIR (\$10)

This 32-bit read/write register is used by the main processor to transfer data to and from the FPCP. The FPCP transfers data through this CIR location in the following cases:

1. Following an evaluate effective address and transfer data primitive
2. Following the read of the register select CIR after a transfer multiple coprocessor registers primitive
3. Following a transfer single main processor register primitive
4. Following a read of an idle or busy format word from the save CIR
5. Following a write of an idle or busy format word to the restore CIR

These five cases are the only times when an access to the operand CIR is legal. At any other time, an access to this CIR location causes a protocol violation.

The FPCP expects all operands that are to be transferred through this CIR location to be aligned with the most significant byte of the register. Any operand larger than four bytes is transferred through this register using a sequence of long-word transfers. If the operand is not a multiple of four bytes in size, the portion remaining after the initial long-word

fers is aligned with the most significant byte of the operand CIR. Figure 7-4 illustrates operand CIR data alignment expected by the FPCP when transferring data through the operand CIR.

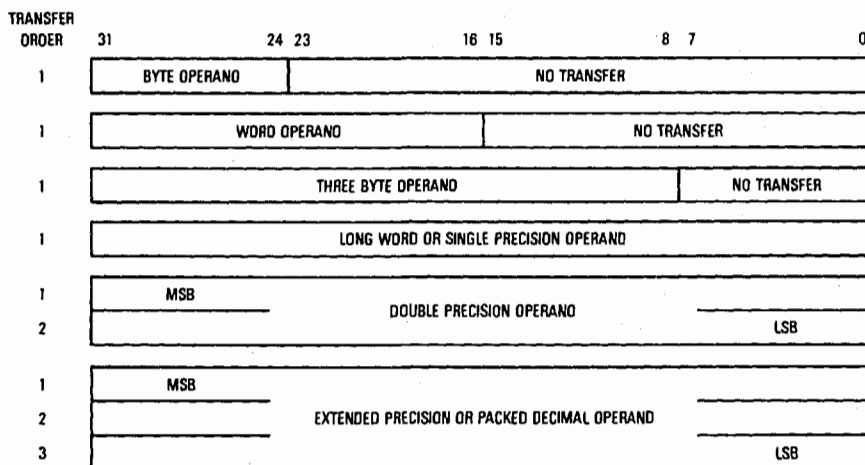


Figure 7-4. Operand CIR Data Alignment

7.2.9 Register Select CIR (\$14)

This 16-bit read-only register is read by the main processor to transfer the register mask from the FPCP during a move multiple floating-point data registers operation. The only time that an access to this register is legal is immediately following the issue of a transfer multiple coprocessor registers primitive to the main processor; at any other time, an access of this CIR location causes a protocol violation.

Although a 16-bit register, the FPCP only utilizes the most significant eight bits; the least significant eight bits are always read as zeros. The most significant eight bits contain the register mask for the multiple register transfer, with each bit set if the corresponding floating-point register is to be transferred. The main processor should not interpret the order of the bits in the register mask, but rather count the number of ones in the mask to determine the number of registers to transfer. Each FPCP floating-point data register is 12 bytes long, and thus requires three long-word transfers.

7.2.10 Instruction Address CIR (\$18)

This 32-bit write-only register is used by the main processor to transfer the address of the FPCP instruction being executed when the PC bit of any primitive is set. The FPCP only sets the PC bit in the first primitive returned during the dialog for an instruction that can cause an exception, or a take pre-instruction exception primitive for a BSUN exception. When the coprocessor is an MC68881, the main processor may optionally transfer the program counter value to the instruction address CIR at that time or ignore the request. (This choice is left to the discretion of the system designer in order to support exception

lers in the most efficient manner.) When the coprocessor is an MC68882, the main processor must transfer the program counter value. The MC68882 issues a protocol violation when the main processor fails to transfer the program counter value. The MPU always transfers the PC when needed.

For the MC68881, accesses to the instruction address CIR are neither expected nor unexpected at any point in an instruction dialog; thus, an access to this CIR location never causes a protocol violation. A write to the instruction address CIR updates the FPIAR register in the MC68881 programming model; a read always returns all ones.

Internally, the MC68882 has three instruction address registers. One register is associated with each of the stages of the MC68882 pipeline (BIU, CU, and APU). The instruction address register associated with the arithmetic processing unit (APU) is the floating-point instruction address register (FPIAR). Since only the APU can report an exception, the FPIAR always points to the instruction that causes the exception whenever an exception occurs. When the instruction address CIR is written (whenever the program counter value is passed), the program counter value is also written to the instruction address register associated with the bus interface unit (BIU) stage of the pipeline. The MC68882 interprets this program counter value as the address of the instruction currently in the BIU. The instruction and its address are moved up the pipeline until the instruction reaches the APU stage of the pipeline. If that instruction causes an exception, its address is in the FPIAR (since the FPIAR is the instruction address register for the APU). This implementation is necessary to ensure that an exception handler can point to the correct instruction, the one that causes the exception. However, this implementation requires that the instruction address CIR be written whenever the MC68882 requests it. The MC68882 issues a protocol violation whenever the main processor fails to supply the requested program counter value. A read of the instruction address CIR always returns all ones.

7.2.11 Operand Address CIR (\$1C)

This 32-bit read/write register is used by the main processor to transfer an operand address in response to the evaluate and transfer effective address or take address and transfer data primitives. Since the FPCP does not utilize either of these primitives, this CIR is not required for operation and is not implemented. An access to this CIR location does not cause a protocol violation; read cycles always return all ones, and the data is ignored during write accesses.

7.3 INTERPROCESSOR TRANSFERS

All interprocessor transfers are initiated by the MPU. During the processing of an FPCP instruction, the MPU transfers instruction information and data to the FPCP using standard M68000 write bus cycles. The MPU also receives data, requests for service, and status information from the FPCP using standard M68000 read bus cycles. A detailed description of the electrical characteristics of the FPCP bus interface is contained in **SECTION 10 BUS OPERATION** and **SECTION 12 ELECTRICAL SPECIFICATIONS**.

7.4 COPROCESSOR INSTRUCTIONS

FPCP instructions are from one to eight words in length. The first word of the instruction is called the operation word, and the second word of the instruction is called the coprocessor command word. Additional words specify the operands and are either extensions

ie effective addressing mode specified in the operation word or immediate operands are part of the instruction. The general format of an FPCP instruction is illustrated in Figure 7-5.

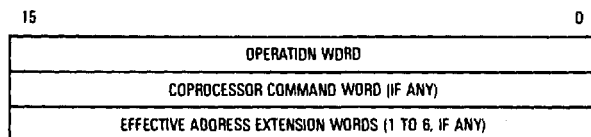


Figure 7-5. Coprocessor Instruction General Format

All coprocessor operations are based on the F-line operation codes (i.e., operation words with bits [15:12]=\$F) which instruct the MPU to call upon a coprocessor for execution of the instruction. Figure 7-6 illustrates the format of this word.

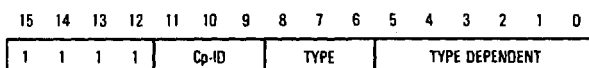


Figure 7-6. FPCP Instruction Operation Word

The Cp-ID field indicates which coprocessor is to be selected. Cp-IDs of 000-101 are reserved by Freescale, and Cp-IDs 110 and 111 are reserved for user definition. The Freescale MPU and FPCP assembler supplies 001 as the Cp-ID for FPCP instructions by default. The type field indicates to the MPU which type of coprocessor operation is selected: general, branch, conditional, save, or restore. The type and type-dependent fields and the coprocessor command word for all FPCP instructions are described in **4.7 INSTRUCTION ENCODING DETAILS**.

7.4.1 Instruction Protocol

All FPCP instructions have a typical protocol which the MPU and FPCP use. This communication protocol is as follows:

1. When the MPU detects an F-line operation word, communication is initiated by writing information (a command, condition selector, or restore format word) to the appropriate FPCP coprocessor interface register location. (The FPCP save instruction is initiated by a read operation.)
2. The MPU then reads the coprocessor response to the previous write operation. The response may indicate any of the following:
 - a. The FPCP is busy. MPU checks for interrupts, processes them if any are pending, and then queries the coprocessor again. This allows synchronizing the main processor and coprocessor.
 - b. An exception condition exists, and the FPCP instructs the MPU to take an exception using a specific exception vector. The MPU acknowledges the exception and initiates exception processing.

- c. There is an FPCP service request (for example, to evaluate the effective address and transfer data between the effective address and the FPCP). The FPCP may also request that the MPU query the coprocessor after the service is performed.
- d. The MPU is not needed for further processing of the coprocessor instruction. Communication is terminated, and the MPU is free to begin execution of the next instruction. If the MPU is in the trace mode, the MPU does not take the trace exception until the FPCP completes the processing of the coprocessor instruction.

Each FPCP instruction type has specific requirements based upon this simplified protocol. The main processor requests required for each FPCP instruction are described in **4.7 INSTRUCTION ENCODING DETAILS**. All FPCP main processor service requests (response primitives) are described in the following paragraphs. In addition, the dialog used by the MPU and the FPCP during the execution of each instruction is detailed in **7.5 INSTRUCTION DIALOGS**.

7.4.2 Response Primitives

Data read from the FPCP coprocessor interface response register is referred to as a primitive. Although the M68000 Family coprocessor interface defines 18 response primitives, the FPCP only uses six of those primitives. For additional information on the complete set of response primitives and how they are serviced; refer to the appropriate processor user's manual. The following paragraphs summarize all FPCP response primitives and how they are used.

The M68000 coprocessor response primitives are encoded in a 16-bit word that is transferred to the main processor through the response CIR. Figure 7-7 illustrates the general format of a response primitive.

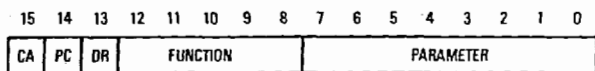


Figure 7-7. M68000 Coprocessor Response Primitive General Format

The encoding of bits [12-0] of a coprocessor response primitive is dependent on the individual primitive being implemented. Bits [15-13], however, are used to specify particular attributes of the response primitive that can be utilized in most of the primitives defined for the M68000 coprocessor interface.

Bit [15] in the primitive format, denoted by CA, is used to specify the come-again operation of the main processor. Whenever the main processor receives a response primitive from the FPCP with the CA bit set to one, it should perform the service indicated by the primitive and then return to read the response CIR again.

Bit [14] in the primitive format, denoted by PC, is used to specify the pass-program-counter operation. If the main processor reads a primitive from the FPCP that has the PC bit set, the main processor should immediately pass the current value of the program counter to the instruction address CIR as the first operation when servicing the primitive request. The value of the program counter passed from the main processor is usually the address of the operation word of the coprocessor instruction executing when the primitive is received.

s always the case if the main processor is the MPU.) The FPCP always sets the PC the first primitive of a general type instruction that might cause an exception (i.e., all of the arithmetic and move single floating-point data register instructions when exceptions are enabled), or the take pre-instruction exception primitive for a BSUN trap during a conditional instruction. By updating the FPIAR in this manner, the FPCP can release the main processor for concurrent execution after all operands are fetched, and exception handlers can later locate an instruction that causes an exception. It should be noted that the PC bit is set in only one primitive response during any instruction dialog, and that the MC68881 does not issue a protocol violation if the main processor ignores the request to transfer the PC. The MC68882 issues a protocol violation if the main processor fails to transfer the PC in response to a request.

Bit [13] in the primitive format, denoted by DR, is the direction bit; it controls the direction of operand transfers between the main processor and the FPCP. If DR is zero, the direction of the transfer is from the main processor to the FPCP (a main processor write). If DR is one, the direction of the transfer is from the FPCP to the main processor (a main processor read). If the operation indicated by a given response primitive does not involve an explicit operand transfer, the value of this bit is dependent on the particular primitive encoding.

NOTE

All primitives issued by the MC68881, with the exception of the null primitive, have the CA bit equal to one, causing the MPU to check the response CIR after any service is performed. This allows the MC68881 to assure correct internal operation and to report exceptions immediately after a service is performed. However, the MC68882 may occasionally issue an evaluate <ea> and transfer data primitive with CA equal to zero. This is done in cases where internal operations are not adversely affected by the omission of the read of the response CIR after the operand transfer.

The following paragraphs describe the response primitive encodings used by the FPCP and the expected main processor response to each one in detail.

7.4.2.1 NULL PRIMITIVE. This primitive is used by the FPCP to synchronize operation with the main processor and to allow concurrent execution by the main processor. The format of the null primitive is shown in Figure 7-8. In addition to the variable bits CA and PC previously discussed, the null primitive uses three other variable bits to identify the required action to be taken by the main processor. Bit [8], denoted by IA, is used to specify that the main processor may process pending interrupts if necessary. Bit [1], denoted by PF, is used to indicate the status of the FPCP during concurrent instruction execution. If the PF bit is zero, the FPCP is executing an instruction; otherwise it is idle. Bit [0], denoted by TF, is used to communicate the true or false result of a conditional evaluation. If TF equals one, the condition is true; otherwise it is false.

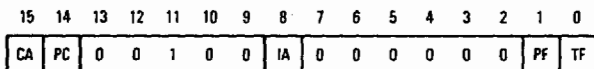


Figure 7-8. Null Primitive Format

indicated by the format of this primitive, there are 32 possible null primitive encodings of which the FPCP uses only seven. Table 7-3 lists the FPCP null primitive encodings and the circumstances in which they are used.

Table 7-3. Null Primitive Encodings

CA	PC	IA	PF	TF	Usage
0	0	0	0	x	Returned by the FPCP in response to the write of a conditional predicate to the condition CIR. The TF bit indicates the result of the conditional evaluation; TF = 1 if the condition is true; TF = 0 if the condition is false.
0	0	0	1	0	Returned when the FPCP is in the idle state. The PF bit indicates that no instruction is being executed; thus, there is no expected response to this primitive.
0	0	1	0	0	Returned when the FPCP enters the middle or end phase of an instruction to allow concurrent execution by the main processor. The CA bit indicates that no further service is required of the main processor, but the PF bit indicates that the FPCP has not completed execution of the instruction. The IA bit indicates that if the main processor is in the trace mode, it may process interrupts while waiting for the FPCP to complete execution of the instruction. Since this primitive does not request any specific service, there is no expected response from the main processor.
0	1	1	0	0	The same as the preceding response, except that the main processor is requested to pass the current program counter before proceeding with the next instruction. This response is returned only as the first response of a dialog.
1	0	1	0	0	Returned when the FPCP is executing an instruction and requires further service from the main processor before the next instruction can be executed. This response is also used when a new FPCP instruction is initiated while a previous one is still being executed. The expected response is for the main processor to re-read the response CIR (after servicing pending interrupts).
1	1	1	0	0	The same as the preceding response, except that the main processor is requested to pass the current program counter before processing any pending interrupts and re-reading the response CIR. This response is returned only as the first response of a dialog.

The meanings of the CA and PC bits are as previously described. If IA equals one, the main processor can process pending interrupts as part of the service for the null primitive; otherwise, interrupts should be ignored. The IA bit is set to a one by the FPCP for most null responses thus allowing the main processor to process pending interrupts anytime that it is "waiting" on the FPCP.

The PF bit indicates the processing state of the FPCP during concurrent instruction execution. In normal operation, the PF bit is of no concern to the main processor. However, if the main processor is in the trace mode, it should wait until the FPCP has completed execution of an instruction before taking the trace exception. By monitoring the PF bit in the null response primitive, the main processor can synchronize with the FPCP in this case. If PF equals zero, the FPCP is executing an instruction; otherwise, it is idle.

The TF bit applies only to the conditional instructions. When the main processor writes a conditional predicate to the condition CIR, the FPCP uses the null primitive to return the true or false result of the conditional evaluation. If TF equals one, the condition is true; otherwise, it is false. For all reads of the response CIR for other instruction types, TF is a don't care bit.

2. EVALUATE EFFECTIVE ADDRESS AND TRANSFER DATA PRIMITIVE. This primitive is used by the FPCP to request the transfer of a data item between the floating-point data and control registers and an external location (either memory or a main processor register). The format of this primitive is shown in Figure 7-9. The main processor services this request by evaluating the effective address indicated by the F-line word of the instruction and transferring the number of bytes indicated by the length field of the primitive to or from the operand CIR.

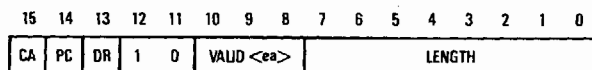


Figure 7-9. Evaluate Effective Address and Transfer Data Primitive Format

Note that the FPCP returns this primitive only once during an instruction dialog. When this primitive is read from the response CIR, it is discarded by the FPCP, and the response encoding is changed to the null primitive. By doing this, the FPCP avoids spurious service request primitives in systems where the MPU is not the main processor.

The meanings of the CA and PC bits are as previously described. The DR bit indicates the direction of data transfer between the effective address location and the operand CIR of the coprocessor. If DR equals zero, the operand is transferred from the effective address location to the coprocessor. If DR equals one, the operand is transferred from the coprocessor to the effective address location.

The effective address that is to be evaluated is specified in the F-line operation word, and any required extension words are fetched by the main processor as needed. If the predecrement or postincrement addressing mode is used, the address register is decremented or incremented before or after the transfer by the size of the operand, as indicated in the length field.

The valid EA field specifies various classes of addressing modes with the encodings shown in Table 7-4. If the effective address in the operation word is not of the specified class, the main processor should write an abort to the control CIR and take an F-line emulator trap. The addressing categories in Table 7-4 are as defined for all M68000 Family processors.

The number of bytes transferred to or from an effective address location is indicated in the length field. If the effective address is a main processor register (register direct), then only lengths of one, two, or four bytes are used. If the effective addressing mode is immediate, the length is always one or even, and the transfer is effective address to coprocessor. If the effective address is a memory location, any length is legal (including odd). If the effective address mode is predecrement or postincrement, with A7 as the specified register and a length of one, the transfer causes the stack pointer to be decremented or incremented by two in order to keep the stack aligned on a word boundary.

Table 7-5 lists the encodings of the evaluate effective address and transfer data primitive that are used by the FPCP and the cases for which they are used.

Table 7-4. Coprocessor Valid Effective Address Codes

000	Control Alterable
001	Data Alterable
010	Memory Alterable
011	Alterable
100	Control
101	Data
110	Memory
111	Any Effective Address

Table 7-5. Evaluate Effective Address and Transfer Data Primitive Encoding

Usage	CA	PC	DR	Valid <ea>	Length
F<op><ea>,FPn (OPCLASS 010) Issued as the first primitive of an instruction dialog to request the transfer of an operand from memory or a main processor data register to the FPCP. The length field indicates the size of the operand: byte, word, long or single, double, extended, or packed BCD.	1 1 ** ** **	* * * * *	0 0 0 0 0	101 101 101 110 110	1 2 4 8 12
FMOVE FPM,<ea> (OPCLASS 011) Issued after the conversion from the internal extended precision format to the destination format is completed to request the transfer of an operand from the FPCP to memory or a main processor data register. The length field indicates the size of the operand: byte, word, long or single, double, extended, or packed BCD.	1 1 ** ** **	0 0 0 0 0	1 1 1 1 1	001 001 001 010 010	1 2 4 8 12
FMOVE <ea>,FPcr and FMOVEM <ea>,FPcr_list (OPCLASS 100) Issued as the first primitive of an instruction dialog to request the transfer of one or more control registers from memory or a main processor register to the FPCP. The length field indicates the total size of all control registers to be moved, 4 bytes per register.	1 1 1 1	0 0 0 0	0 0 0 0	111 101 110 110	4 4 8 12
FMOVE FPcr,<ea> and FMOVEM FPcr_list,<ea> (OPCLASS 101) Issued as the first primitive of an instruction dialog to request the transfer of one or more control registers from the FPCP to memory or a main processor register. The length field indicates the total size of all control registers to be moved, 4 bytes per register.	1 1 1 1	0 0 0 0	1 1 1 1	011 001 010 010	4 4 8 12

*PC=1 if any arithmetic exceptions are enabled; otherwise PC=0.

**CA=0 for some MC68882 instructions with S,D,X instruction operand formats; otherwise, CA=1.

7.4.2.3 TRANSFER SINGLE MAIN PROCESSOR REGISTER PRIMITIVE. This primitive is used by the FPCP to request the transfer of one main processor register. The format of this primitive is shown in Figure 7-10. The main processor services this request by writing a long word to the operand CIR.

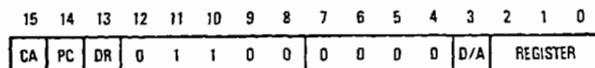


Figure 7-10. Transfer Single Main Processor Register Primitive Format

primitive is only utilized for the move multiple floating-point data register instruction if the register list is specified as dynamic. Therefore, when this primitive is issued by the FPCP (to fetch the register list), CA is always set; DR, PC, and D/A are always clear (D/A identifies the selected register as a data or address register; zero indicates it is a data register). The least significant three bits identify the main processor data register that contains the register list.

Note that the FPCP returns this primitive only once during an instruction dialog. When this primitive is read from the response CIR, it is discarded by the FPCP, and the response encoding is changed to the null primitive until the request has been serviced. By doing this, the FPCP avoids spurious service requests in systems where the MPU is not the main processor.

7.4.2.4 TRANSFER MULTIPLE COPROCESSOR REGISTERS PRIMITIVE. This primitive is used by the FPCP to request the transfer of a list of floating-point data registers to or from memory. The format of this primitive is shown in Figure 7-11. The main processor services this request by performing an implied effective address evaluation, reading a register list from the register select CIR, and transferring the selected registers (where each register is the size indicated by the length field of the primitive) between the operand CIR and memory. The MPU uses long-word transfers whenever possible while servicing this primitive.

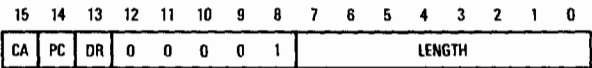


Figure 7-11. Transfer Multiple Coprocessor Registers Primitive Format

Note that the FPCP returns this primitive only once during an instruction dialog. When this primitive is read from the response CIR, it is discarded by the FPCP, and the response encoding is changed to the null primitive until the request has been serviced. By doing this, the FPCP avoids spurious service requests in systems where the MPU is not the main processor.

The meanings of the CA and PC bits are as previously described. For this primitive, CA is always set, and PC is always clear (since the FMOVEM instruction cannot cause an exception). The DR bit indicates the direction of the transfer between the effective address location and the operand CIR. If DR equals zero, the listed registers are transferred from the effective address location to the FPCP. If DR equals one, the listed registers are transferred from the FPCP to the effective address location.

The length field indicates the size, in bytes, of each register to be transferred; for the FPCP, the length is always 12 bytes. The register list that is read from the register select CIR is used by the main processor to determine the number of registers to be transferred (but not the order of the transfer). For each bit that is set in the 16-bit register list, one operand of the size indicated by the length field is transferred. Thus, the total number of bytes transferred in response to this primitive is the product of the length and the number of ones in the register list. Since the FPCP has only eight floating-point data registers, the register list always has zeros in the least significant byte.

The effective address that is evaluated by the main processor is specified in the F-line operation word, and any required extension words are fetched by the main processor, as

ad. If the predecrement or postincrement addressing mode is used, the address register is decremented or incremented (before or after the transfer) by the size of the operand as indicated by the length field. The effective addressing modes that are valid for this primitive are determined by the DR bit, and the mode is validated by the main processor. If DR equals zero, the control and postincrement addressing modes are allowed. If DR equals one, the control alterable and predecrement addressing modes are allowed. If the effective address field of the operation word is not valid for the selected multiple register transfer, the main processor writes an abort to the control CIR (before reading the register select CIR) and takes an F-line trap.

For the control and postincrement addressing modes, the registers are transferred using ascending addresses. For the postincrement addressing mode, the address register is incremented by the length value after each register is transferred. Thus, the final value of the address register is the initial value plus the total number of bytes transferred during the primitive execution.

For the predecrement addressing mode, the operands are written to memory with descending addresses, but the bytes within each operand are written to memory with ascending addresses. For example, Figure 7-12 illustrates the transfer of two floating-point data registers to a stack, using the $-(An)$ addressing mode. The designated stack pointer is decremented by 12 bytes before the transfer of each register. Then the bytes within each register are written to memory with ascending addresses. Thus, the address register is decremented by the total number of bytes transferred by the end of the primitive execution.

7

7.4.2.5 TAKE PRE-INSTRUCTION EXCEPTION PRIMITIVE. Take exception primitives are used by the FPCP to instruct the main processor to abort the current operation and initiate exception processing. The main processor services these requests by writing an exception acknowledge to the control CIR (which clears the pending exception in the FPCP), by creating the appropriate stack frame on the currently active supervisor stack, and by beginning execution of an exception handler. The exception handler is located by using the vector number that is supplied as part of the take exception primitive. Table 7-6 lists the vector numbers used by the FPCP.

Note that the MC68881 returns one of these primitives only once during the instruction dialog. When an exception acknowledge is written to the control CIR, the take exception primitive is discarded by the MC68881, and the response encoding is changed to the null primitive. By doing this, the MC68881 assures that the take exception request is received

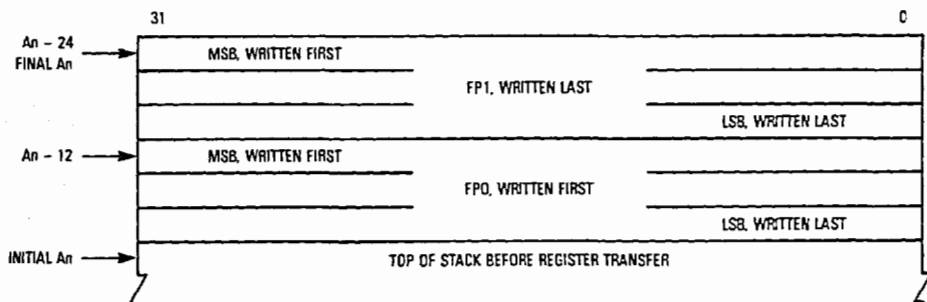


Figure 7-12. Transfer Multiple Floating-Point Data Register to Stack Example

Table 7-6. FPCP Vector Numbers

Vector Decimal	Number Hexadecimal	Vector Offset (Hexidecimal)	Assignment
11	\$0B	\$02C	F-Line Emulator
13	\$0D	\$034	Coprocessor Protocol Violation
48	\$30	\$0C0	Branch or Set on Unordered Condition
49	\$31	\$0C4	Inexact Result
50	\$32	\$0C8	Floating-Point Divide by Zero
51	\$33	\$0CC	Underflow
52	\$34	\$0D0	Operand Error
53	\$35	\$0D4	Overflow
54	\$36	\$0D8	Signaling NAN

by the main processor, but avoids spurious service request primitives in systems where the MPU is not the main processor.

The MC68882, however, does not discard the primitive unless a read of the save CIR is detected. This is to ensure that the FSAVE instruction is executed and the state of the conversion unit (CU) is saved. The state of the CU must be saved because a second instruction, in the CU, may be partially executed.

While the M68000 coprocessor interface defines three take-exception primitives, the FPCP utilizes only two of them. The other take exception primitive is described in the next section.

The take pre-instruction exception primitive is used by the FPCP when an arithmetic (OP-CLASS 000, 010, and 011) or conditional instruction is initiated and an exception is pending from a previously executed, concurrent instruction. This primitive is also returned if an illegal command word is written to the command CIR or if a protocol violation occurs. Finally, this primitive is issued when a conditional instruction is executed that utilizes one of the IEEE nonaware conditional predicates, and the NAN bit in the FPSR condition code byte is set. The format of this primitive is shown in Figure 7-13.

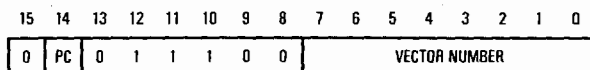


Figure 7-13. Take Pre-Instruction Exception Primitive Format

The CA bit is always zero for this primitive, since there is an implied protocol pre-emption in this service request. The PC bit is zero if the exception is pre-empting the execution of a new FPCP instruction. The PC bit is one if the exception is due to an illegal command word, or if it is reported during the execution of a conditional instruction in lieu of the true/false result of the conditional evaluation. The vector number identifies the type of the exception and is used by the main processor to locate the exception handler routine.

In response to this primitive, the MPU creates a four-word stack frame on top of the currently active supervisor stack. The format of this stack frame is shown in Figure 7-14. The value of the program counter in the stack frame is the address of the F-line operation word of

FPCP instruction that was preempted by the exception (i.e., the arithmetic or conditional instruction attempted in the case of an exception pending from a previous instruction or an F-line exception, or the conditional instruction in the case of a BSUN exception). Thus, if no modifications are made to the stack frame within the exception handler, an RTE instruction causes the MPU to return and reinitiate the instruction that was being attempted when the primitive was received. Refer to the appropriate user's manual for further details on exception handling by the MPU.

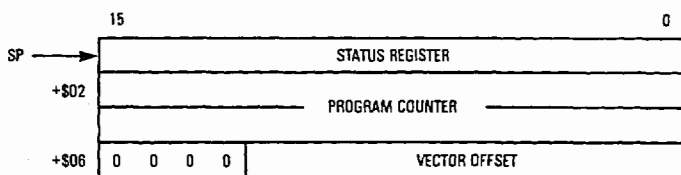


Figure 7-14. Pre-Instruction Exception Stack Frame

7

7.4.2.6 TAKE MID-INSTRUCTION EXCEPTION PRIMITIVE. This primitive is used by the FPCP when an exception occurs during the execution of an FMOVE Fp_n,<ea> instruction. In the MC68882, an exception caused by a previous instruction is reported by the current instruction using this primitive. See 7.4.2.5 TAKE PRE-INSTRUCTION EXCEPTION PRIMITIVE for information common to both take exception primitives. The format of this primitive is shown in Figure 7-15.

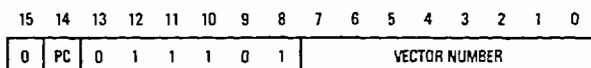


Figure 7-15. Take Mid-Instruction Exception Primitive Format

The CA bit is always zero for this primitive, because there is an implied protocol preemption in this service request. The PC bit is always zero, since a null primitive earlier in the dialog for the move-out instruction is used to request the program counter transfer. The vector number identifies the type of the exception, and is used by the main processor to locate the exception handler routine.

In response to this primitive, the MPU creates a ten-word stack frame on top of the currently active supervisor stack. The format of this stack frame is shown in Figure 7-16. If the exception is due to an FMOVE Fp_n,<ea> instruction, the ScanPC value is the address of the instruction immediately following the FMOVE instruction. The value of the program counter in the stack frame is the address of the F-line operation word of the FPCP instruction that caused the exception. The operation word image contains the F-line word of the FMOVE instruction. The effective address value is the memory address of the destination operand. Note that the take mid-instruction exception primitive is used in this case solely for the purpose of placing the evaluated effective address in the stack frame, to avoid requiring an exception handler to recalculate it.

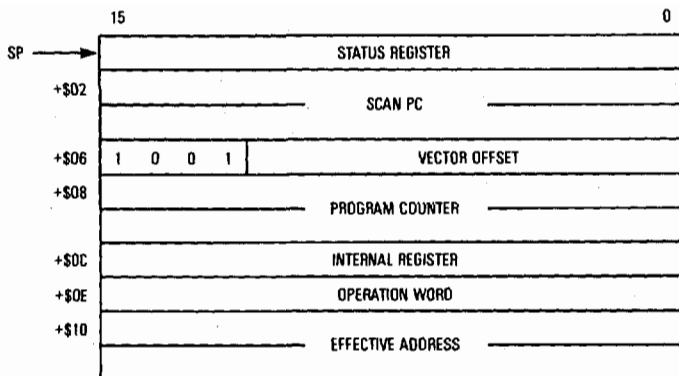


Figure 7-16. Mid-Instruction Stack Frame

If the exception is caused by a previous instruction, the ScanPC value is the address of the instruction immediately following the instruction that reported the exception. The value of the PC in the stack frame is the address of the F-line operation word of the instruction that reported the exception. The operation word image contains the F-line word of the instruction that reported the exception. The effective address only contains the valid effective address when an evaluate <ea> primitive was issued before the exception was reported.

If no modifications are made to the stack frame within the exception handler, an RTE instruction causes the MPU to return to read the response CIR. Thus, the main processor continues the execution of the instruction upon return.

7.4.2.7 RESPONSE PRIMITIVE SUMMARY. Table 7-7 lists in numeric order a summary of all primitive responses utilized by the FPCP.

7.5 INSTRUCTION DIALOGS

The following paragraphs describe in detail the coprocessor communications dialogs that are executed by the FPCP and MPU during each floating-point instruction. In this discussion, a dialog refers to the sequence of command and data transfers to the FPCP, and the service request primitives that are returned to control that sequence. Although the following discussion assumes that the main processor is an MC68020 or MC68030, information is also presented that may be used by designers of systems that utilize a different main processor.

The diagrams presented in the following paragraphs represent the activity of the MPU and the FPCP during the execution of a floating-point instruction. In these diagrams, boxes are used to identify periods of time during which a device is actively participating in the execution of an instruction; the absence of a box during a period indicates that a device is waiting on the other one to complete an operation, or that concurrent execution of unrelated instructions may take place.

Each box in the following diagrams is labeled to indicate the activity depicted by that box. The labels above the boxes identify the actions taken by the main processor, and the labels below the boxes identify the encoding of the response CIR at any time during a dialog.

Table 7-7. MC68881/MC68882 Primitive Responses

Primitive Value	Primitive Type	Comments
\$0800 \$0801 \$0802 \$0900	Null	CA = 0, PC = 0, IA = 0, PF = 0, TF = 0 CA = 0, PC = 0, IA = 0, PF = 0, TF = 0 CA = 0, PC = 0, IA = 0, PF = 1, TF = 0 CA = 0, PC = 0, IA = 1, PF = 0, TF = 0
\$1504 \$1608 \$160C	Evaluate <ea> and Transfer Data CA = 0, PC = 0, DR = 0 (External to MC68882)	Single Double Extended
\$1C08 \$1C31 \$1C32 \$1C33 \$1C34 \$1C35 \$1C36	Take Pre-Instruction Exception PC = 0	F-Line Emulator Inexact Result Floating-Point Divide by Zero Underflow Operand Error Overflow Signaling NAN
\$1D0D \$1D31 \$1D32 \$1D33 \$1D34 \$1D35 \$1D36	Take Mid-Instruction Exception PC = 0	Coprocessor Protocol Violation Inexact Result Floating-Point Divide by Zero Underflow Operand Error Overflow Signaling NAN
\$3104 \$3208 \$320C	Evaluate <ea> and Transfer Data CA = 0, PC = 1, DR = 1 (MC68882 to External)	Single Double Extended
\$4900	Null	CA = 0, PC = 1, IA = 1, PF = 0, TF = 0
\$5504 \$5608 \$560C	Evaluate <ea> and Transfer Data CA = 0, PC = 1, DR = 0 (External to MC68882)	Single Double Extended
\$5C30	Take Pre-Instruction Exception PC = 0	Branch or Set On Unordered
\$810C	Transfer Multiple Coprocessor Registers CA = 1, PC = 0, DR = 0 (Memory to FPCP)	
\$8900	Null	CA = 1, PC = 0, IA = 1, PF = 0, TF = 0
\$8C00 \$8C01 \$8C02 \$8C03 \$8C04 \$8C05 \$8C06 \$8C07	Transfer Single Main Processor Register CA = 1, PC = 0, DR = 0 (Main Processor to FPCP)	D0 D1 D2 D3 D4 D5 D6 D7
\$9501 \$9502 \$9504 \$9608 \$960C \$9704	Evaluate <ea> and Transfer Data CA = 1, PC = 0, DR = 0 (External to FPCP)	Byte Word Long, Single, FPCR, or FPSR Double or Two FPCR's (Memory Only) Extended, Packed, or Three FPCR's (Memory Only) FPIAR
\$A10C	Transfer Multiple Coprocessor Registers CA = 1, PC = 0, DR = 1 (FPCP to Memory)	
\$B101 \$B102 \$B104 \$B208 \$B20C \$B304	Evaluate <ea> and Transfer Data CA = 1, PC = 0, DR = 1 (FPCP to External)	Byte Word Long, Single, FPCR, or FPSR Double or Two FPCR's (Memory Only) Extended, Packed or Three FPCR's (Memory Only) FPIAR

Table 7-7. MC68881/MC68882 Primitive Responses (Continued)

Primitive Value	Primitive Type	Comments
\$C900	Null	CA = 1, PC = 1, IA = 1, PF = 0, TF = 0
\$CC00 \$CC01 \$CC02 \$CC03 \$CC04 \$CC05 \$CC06 \$CC07	Transfer Single Main Processor Register CA = 1, PC = 1, DR = 0 (Main Processor to FPCP)	D0 D1 D2 D3 D4 D5 D6 D7
\$D501 \$D502 \$D504 \$D608 \$D60C	Evaluate <ea> and Transfer Data CA = 1, PC = 1, DR = 0 (External to FPCP)	Byte Word Long or Single Double (Memory Only) Extended or Packed (Memory Only)

When a response CIR encoding is indicated, that encoding is received by the main processor any time that the response CIR is read until the next primitive encoding is indicated.

In all of the succeeding paragraphs, the following assumptions are made:

1. Before the start of an instruction dialog, except for the FSAVE and FRESTORE instructions, the FPCP is in the idle state.
2. The MPU and the FPCP communicate via a 32-bit data bus.
3. The memory width is 32 bits, and all memory operands are long-word aligned.

Also, for periods during which the MPU is required to wait for the FPCP (i.e., during move-to-memory operation, or if the MPU is in the trace mode), only one of the response CIR reads is explicitly indicated. In actual operation, numerous reads of the response CIR may occur in these cases. Similarly, if the FPCP is not idle before the initiation of a new instruction, multiple reads of the null (CA = 1, IA = 1; \$8900) primitive may occur after the command or condition CIR write and before the read of the first primitive shown in a diagram.

7.5.1 General Instructions

This group of instructions includes all of the arithmetic instructions, the move system control register instructions, the move instructions, and the move multiple floating-point register instructions. The factor common to these instructions is the format of the F-line operation word, which uses the cpGEN format of the M68000 Family coprocessor instruction set. Thus, the initial phase of the communications dialog for these instructions is identical, with the MPU writing the command word to the FPCP and then relying on the FPCP to control the remainder of the dialog through the use of the coprocessor interface response primitive set. The following paragraphs discuss the five different protocols that are used by the FPCP for this group of instructions.

For each of the general instruction dialogs, with the exception of the register-to-register dialog, there is an important consideration for systems that use the FPCP with a main processor other than an MPU. This consideration is that the come-again request in any evaluate effective address and transfer data primitive or transfer multiple coprocessor

ers primitive should not be ignored. The FPCP sets the CA bit in these primitives to a correct operation regardless of the frequency relationship between the FPCP clock and the main processor clock. By requiring the main processor to perform a final read of the response CIR (which is a cycle that is synchronous with the FPCP CLK signal) after the last operand CIR access and before continuing with the next instruction, the FPCP assures that the operand transfer is completed internally before the main processor can initiate the next instruction by writing the command or condition CIRs. This sequence assures that spurious protocol violations (detected by the FPCP) do not occur in systems where the main processor clock frequency is much faster than the MC68881 clock frequency.

During the instruction dialogs for external-to-register (opclass 010 and register-to-external (opclass 011)) instructions, the MC68882 in some cases issues the evaluate effective address and transfer data primitive with CA = 0 instead of CA = 1. In these cases, the main processor need not read the response CIR after the coprocessor has transferred the operands for that instruction. This provides more potential instruction overlap with the next coprocessor instruction. Normally, a second coprocessor instruction causes the main processor to write to the command or condition CIR and then to read from the response CIR. If the read from the response CIR of the second instruction occurs earlier than three clocks after the completion of the last operand transfer of the previous instruction, spurious protocol violations may occur. In a worst-case situation, the main processor uses these three clocks in writing to the command CIR. However, if the main processor has a higher clock frequency than the MC68882, it is possible that the write to the command CIR can take less than three MC68882 clocks. The design of the MC68882 allows the MPU clock frequency to be as much as 1.5 times the MC68882 clock frequency. For main processors other than the MC68020 or MC68030, the system designer must ensure that the main processor does not read the response CIR during the initiation of an instruction less than three MC68882 clocks after the last operand transfer of the previous instruction.

7.5.1.1 REGISTER-TO-REGISTER (OPCLASS 000). This dialog is utilized for all of the arithmetic and move instructions that use floating-point data registers for both the source and destination operands and for the FMOVECR instruction. Since the FPCP contains both operands when such an instruction is initiated, no external data references are required before the calculation can be performed. Thus, after the MPU has written the command word to the FPCP, it is released to execute the next instruction. If any arithmetic exceptions are enabled, the FPCP requests the transfer of the program counter. This request can be ignored by a main processor using an MC68881, but the MC68882 issues a protocol violation if the main processor ignores this request. The program counter write cycle does not affect instruction execution time (since it occurs concurrently with the FPCP instruction execution).

The FPCP dialog for this instruction type is shown in Figure 7-17. Also shown in this figure is the key for all of the dialog figures presented in subsequent paragraphs.

7.5.1.2 EXTERNAL-TO-REGISTER (OPCLASS 010). This dialog is utilized for all of the arithmetic and move instructions that reference memory or a main processor register for the source operand. Since the FPCP does not contain both operands when such an instruction is initiated, external data references are required before the calculation can be performed. The FPCP requests the fetch of the required external operand with the first primitive of the dialog. The second primitive of the dialog is then used to release the main processor to execute the next instruction (once the operand transfer is completed). Note that the read of the first primitive causes the response CIR encoding to be changed to the null primitive,

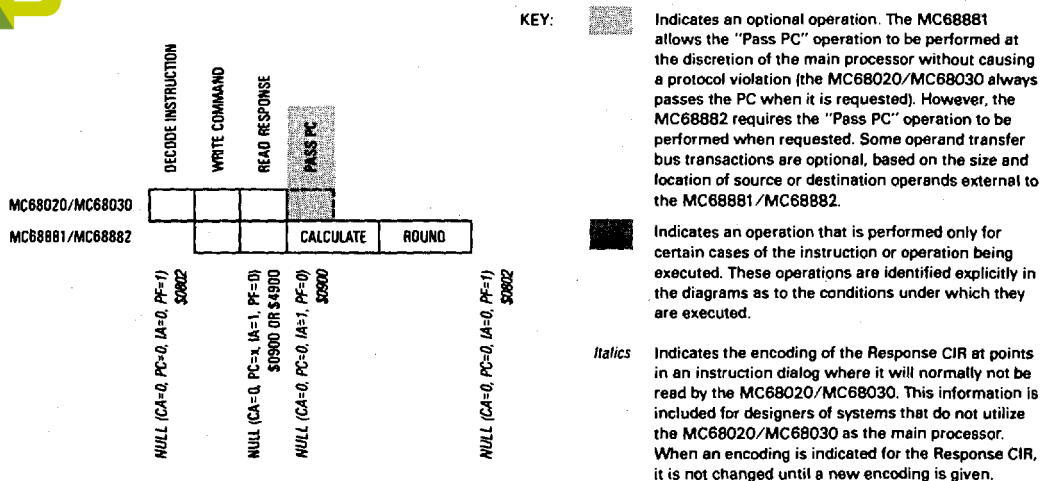


Figure 7-17. MC68881 Register-to-Register Instruction Dialog

thus avoiding spurious request primitives in non-MPU based systems. The MC68881 dialog for this instruction type, which also applies to the MC68882 with operand data formats other than single, double, or extended, is shown in Figure 7-18.

When the operand data format is single, double, or extended, the MC68882 issues the evaluate <ea> and transfer data primitive with CA=0 instead of CA=1. Figure 7-19 shows the dialog for the MC68882 external-to-register instructions with single, double, or extended data format operands.

If any arithmetic exceptions are enabled, the FPCP requests the transfer of the program counter with the first primitive. However, the main processor using an MC68881 can ignore this request; the MC68882 issues a protocol violation if the main processor ignores the request.

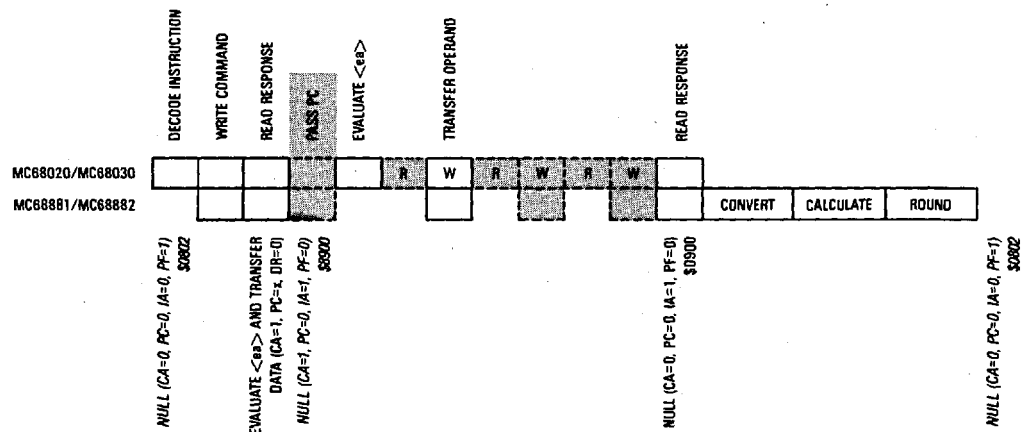


Figure 7-18. MC68881/MC68882 External-to-Register Instruction Dialog



7.5.1.3 REGISTER-TO-EXTERNAL (OPCLASS 011). This dialog is utilized only for the move from floating-point data register instruction. The MC68881 dialog for this instruction type, which also applies to the MC68882 except when the data format is single, double, or extended, is shown in Figure 7-20. The first primitive returned depends on the destination

7.5.1.3 REGISTER-TO-EXTERNAL (OPCLASS 011). This dialog is utilized only for the move from floating-point data register instruction. The MC68881 dialog for this instruction type, which also applies to the MC68882 except when the data format is single, double, or extended, is shown in Figure 7-20. The first primitive returned depends on the destination



format, since additional format information is required to generate a packed decimal operation operand when a dynamic k factor is specified. If the destination data type is packed decimal and a dynamic k factor is used, the first response is the transfer single main processor register primitive (to transfer the contents of the register containing the k factor). For all other destination data formats, the first request is the null (CA=1) primitive, since a conversion from the internal data format to the desired destination format must be performed before any further action is required of the main processor. For the dynamic k-factor case, the conversion starts after the main processor register transfer is completed. The conversion starts immediately after the first read of the response CIR for all other destination operand formats. The main processor is allowed to service pending interrupts while it is waiting for the conversion to complete.

If any arithmetic exceptions are enabled, the first primitive requests the transfer of the program counter. However, this request can be ignored when the main processor uses an MC68881; the MC68882 issues a protocol violation when the main processor ignores this request. The program counter write cycle may not affect instruction execution time (since it can occur concurrently with the operand conversion if the destination format is not packed decimal with a dynamic k factor). Only the first primitive requests the transfer of the program counter; thus, if the transfer single main processor register primitive is issued first, the PC bit is not set in any subsequent null primitive. If the first primitive is the null primitive and the program counter transfer is required, the PC bit is set.

The pass program counter operation is requested in one of two of the primitive encodings (shown in Figure 7-20 with the notation "PC=x"). For the packed decimal with a dynamic k-factor case, the dark shaded operations are always performed with the PC bit set if necessary. The MPU services the transfer single main processor primitive with the PC bit set by first transferring the program counter and then transferring the requested register. For all other destination data formats, the dark shaded operations are not performed, and the box labeled "Convert" is "folded under" the first box labeled "Read Response". For these cases, the null primitive may request the transfer of the PC, and that transfer occurs concurrently with the operand conversion by the FPCP.

When the operand conversion is completed, the next read of the response CIR returns the evaluate effective address and transfer data primitive. The main processor then reads the conversion result from the operand CIR and writes it to the appropriate destination location. Note that the read of the evaluate effective address and transfer data primitive causes the response CIR encoding to be changed to the null primitive, thus avoiding spurious request primitives in non-MC68020 or non-MC68030 based systems.

The operation boxes that are marked "R" or "W" indicate an operand read or write cycle, respectively, by the MPU. Those operand transfer boxes that are lightly shaded are optionally executed, depending on the size and location of the source operand. For example, none of the shaded boxes are executed for destination operands that reside in the MPU registers.

The MC68882 dialog differs from the dialog shown in Figure 7-20 when the operand data format is single, double, or extended, except for the following conditions:

- The data in the floating-point register is data type NAN, unnormalized, or denormalized.
- A rounding overflow or underflow has occurred, and the operand data format is single or double.
- The INEX2 bit of the FPSR exception enable byte is set, and the operand data format is single or double.

...in any of these conditions occurs, the dialog shown in Figure 7-20 applies. Otherwise, ...on the operand data format is single, double, or extended, the MC68882 issues the evaluate <ea> and transfer data primitive with CA=0 instead of CA=1. Figure 7-21 shows the MC68882 dialog.

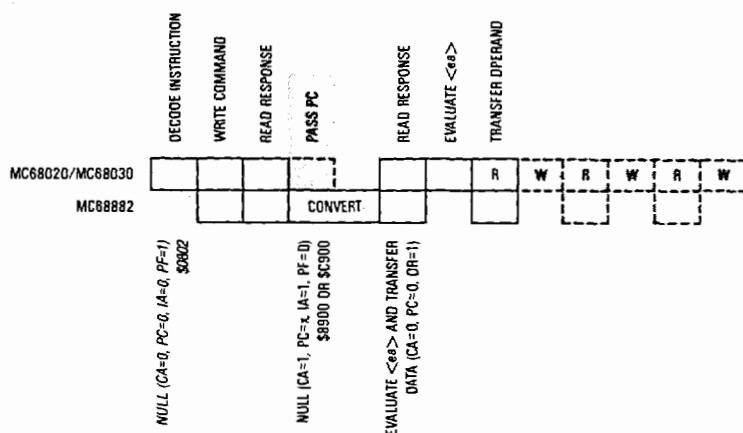


Figure 7-21. MC68882 Register-to-External Instruction Dialog (S, D, and X Formats)

7.5.1.4 MOVE CONTROL REGISTERS (OPCLASS 100 and 101). This dialog is utilized for the move single or multiple floating-point system control registers instructions. The dialog for this instruction type is shown in Figure 7-22. The first primitive of the dialog requests that the main processor evaluate the effective address and transfer the appropriate number of bytes to or from the operand CIR. The read of the first primitive causes the response CIR encoding to be changed to the null primitive, thus avoiding spurious request primitives in non-MC68020 or non-MC68030 based systems. When the transfer data primitive service is complete, the main processor is released to begin execution of the next instruction. Note

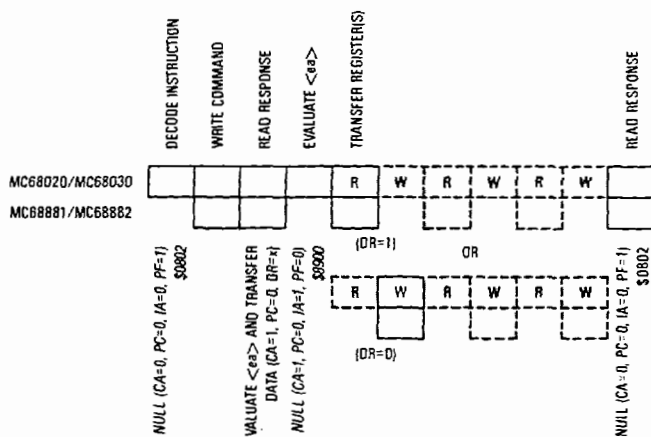


Figure 7-22. Move Control Register Instruction Dialog

since this instruction type cannot cause an exception, the PC bit is not set in any itive; thus, these instructions can be used to read or write the control registers without overwriting the FPIAR contents.

The operation boxes that are marked "R" or "W" indicate an operand read or write cycle, respectively, by the MPU. Those operand transfer boxes that are shaded are optionally executed, depending on the size and location of the source operand. For example, none of the shaded boxes are executed for source operands that reside in the MPU registers.

7.5.1.5 MOVE MULTIPLE FPn (OPCLASS 110 and 111). This dialog is utilized for the move multiple floating-point data registers instruction. The dialog for this instruction type is shown in Figure 7-23. The first primitive of the dialog depends on the type of register list specified by the instruction. If the static register list form of the instruction is used, the first service request issued is the transfer multiple coprocessor registers primitive. For the dynamic register list form, the first primitive requests the transfer of the main processor data register that contains the register mask, and then the transfer multiple coprocessor registers primitive is issued. In response to the transfer multiple coprocessor registers primitive, the main processor reads the register list from the register select CIR and transfers one register for each bit that is set in the list. (Note that the register list can be equal to zero; in which case, no register transfer occurs.)

The read of the transfer single main processor register and transfer multiple coprocessor registers primitives causes the response CIR encoding to be changed to the null primitive, thus avoiding spurious request primitives in non-MC68020 or non-MC68030 based systems. If the transfer single main processor register primitive is issued, the transfer multiple coprocessor register primitive is not issued until the first service request is completed. When the transfer multiple coprocessor register primitive service is complete, the main processor is released to begin execution of the next instruction. Note that since this instruction type cannot cause an exception, the PC bit is not set in any primitive; thus, these

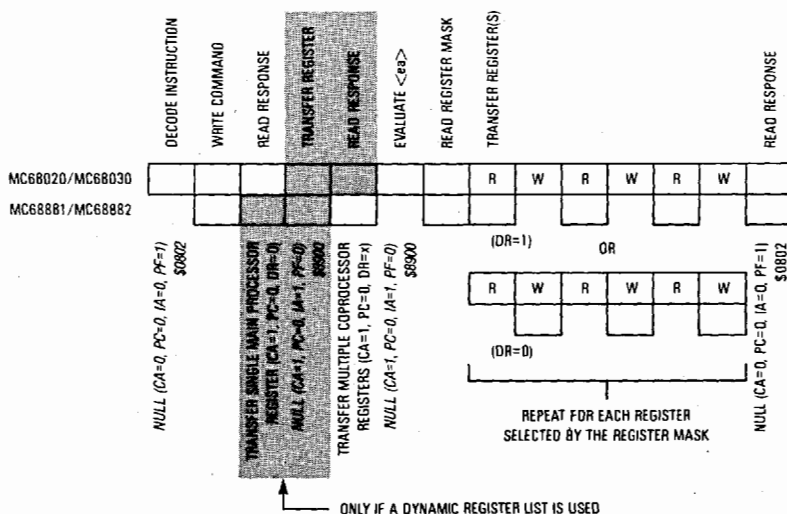


Figure 7-23. Move Multiple Floating-Point Data Registers Instruction Dialog

The operation boxes that are marked “R” and “W” indicate an operand read or write cycle, respectively, by the MPU.

7.5.2 Conditional Instructions

This group of instructions includes the FBcc, FDBcc, FNOP, FScc, and FTRAPcc instructions. These instructions have two factors in common. First, the execution of each instruction is inherent in the M68000 Family coprocessor instruction set definition, and the dialog used for all of them is the same. Second, in each of these instructions, the coprocessor completes all previous floating-point instructions before it begins to evaluate the result of the received conditional predicate. This guarantees a sequential execution model. The dialog begins when the main processor writes the conditional predicate to the FPCP and then reads the response CIR. If the APU and the CU (in the case of an MC68882) are not idle, a null primitive (CA=1, IA=1, PC=0, TF=0) is returned, and the main processor reads the response CIR again later. This process of rereading the response CIR continues until the coprocessor is idle. Note that it is possible for a pre-instruction exception to be reported at any time during this process. If no exception is reported, and when the coprocessor is finally idle, the coprocessor evaluates the condition and responds with a null primitive (CA=0, TF=x, where x is the true or false result). The main processor then proceeds with the appropriate conditional action. The dialog (assuming that the coprocessor is idle when the conditional predicate is written) is shown in Figure 7-24.

7.5.3 Context Switch Instructions

This group of instructions includes the FSAVE and FRESTORE instructions. The factor common to these instructions is that the execution of each one is inherent in the M68000 Family coprocessor instruction set definition, and the coprocessor does not control the

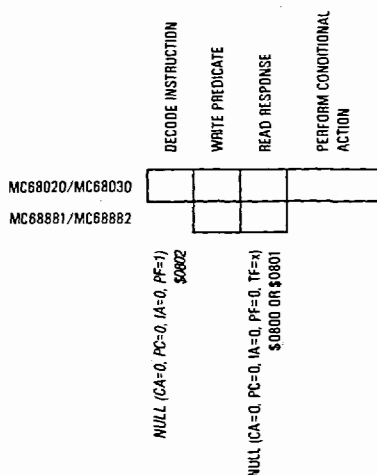


Figure 7-24. Conditional Instruction Dialog

ing in the flexible manner available with the general and conditional instruction types. The dialog consists of the save or restore command, followed by the transfer of the appropriate state frame. The only control that the FPCP has over this dialog is for the FSAVE instruction; in which case, it may request that the main processor delay the save operation until the FPCP is ready to perform it. These dialogs are discussed in the following paragraphs.

7.5.3.1 FSAVE. This dialog is utilized for the context save instruction. The dialog for this instruction is shown in Figure 7-25. There are no primitive responses during this dialog; instead, the FPCP controls the frame transfer to a limited extent through the use of the format word encoding.

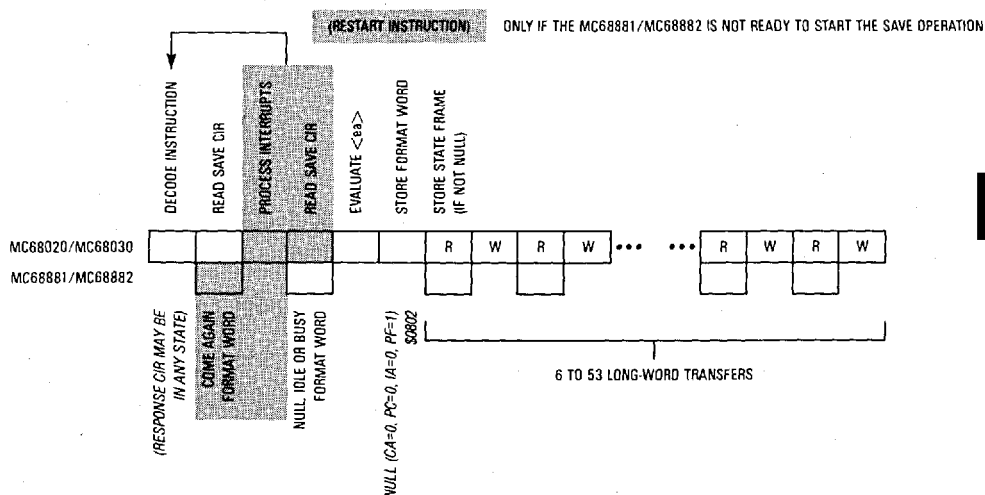


Figure 7-25. FSAVE Instruction Dialog

The main processor initiates this dialog by reading from the save CIR. During this read cycle, the FPCP returns a format word that indicates the current state of the machine. For most cases of this dialog with the MC68881, the first read of the save CIR returns the idle format word, and the main processor then proceeds to transfer six long words from the operand CIR to memory. In this dialog with the MC68882, the idle format word is followed by 14 long words. Optionally, the first primitive may be a null format word, in which case no state frame is transferred. Alternatively, the first primitive may be a busy format word, in which case 45 (53 for the MC68882) long words are transferred. Finally, the save CIR read may return the not-ready, come-again format word. In this case, the main processor may process pending interrupts and restart the instruction, or reread the save CIR until a different format word is received. The invalid format word may also be returned, as discussed in **7.5.4.6 FORMAT EXCEPTION, FSAVE INSTRUCTION**.

After the MPU receives a valid format word, it then evaluates the effective address and writes the format word to that address. The appropriate state frame is then transferred to the effective address, and the main processor is free to proceed with the execution of the

7

7.5.3.2 FRESTORE. This dialog is utilized for the context restore instruction. The dialog for this instruction is shown in Figure 7-26. There are no primitive responses during this dialog; instead, the FPCP controls the frame transfer to a limited extent through the use of the format word encoding.

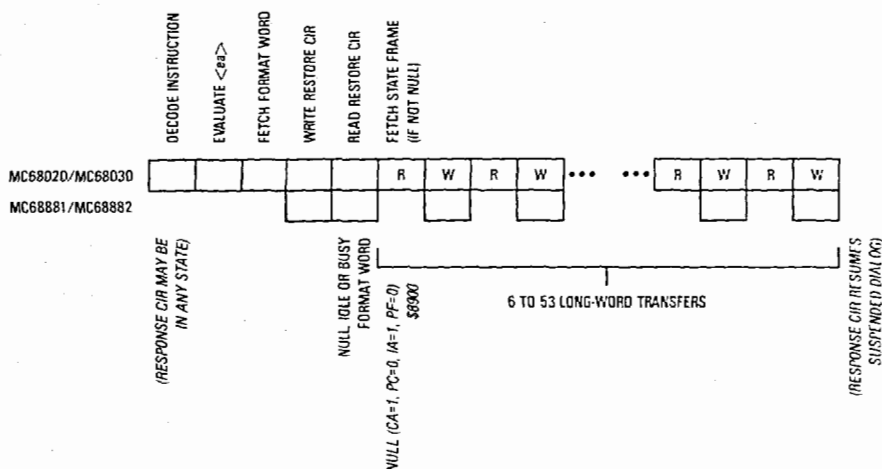


Figure 7-26. FRESTORE Instruction Dialog

The main processor initiates this dialog by evaluating the effective address, fetching a format word from that address, and writing the format word to the restore CIR. The main processor then reads the restore CIR to verify that the format word is valid. During this read cycle, the FPCP returns a format word that indicates if the format word that was written is valid for the current revision of the device. If the format word is valid, the same value that was written is read back from the restore CIR, and the main processor proceeds to transfer the state frame appropriate for the format word. The state frame size is 0, 6, or 45 long words for the current implementation of the MC68881. For the MC68882, the corresponding state frame sizes are 0, 14, and 53 long words. The invalid format word may also be returned as discussed in **7.5.4.7 FORMAT EXCEPTION, FRESTORE INSTRUCTION**.

Note that after the state restore operation is complete, the FPCP is in the state of the instruction that was previously suspended with an FSAVE instruction.

This group of dialogs is actually a set of special cases of the dialogs described previously; they are grouped here for quick reference, and to simplify the preceding discussions. For each of the exception processing dialogs, only the differences from the normal instruction dialogs shown previously are discussed here. Also, it should be noted that these dialogs do not include all exception processing sequences that involve the FPCP; they only include those exceptions that are directly related to the coprocessor interface operation. For example, main processor detected F-line exceptions are not included since no coprocessor interface dialog occurs as part of the exception processing for this type of an exception.

The dialog for the coprocessor protocol violation exception is also omitted from the following diagrams. This is because these exceptions are not expected to occur during the normal operation of a fully debugged system, and the dialog is not readily predictable (either before or after the protocol violation occurs). By definition, the cause of the exception for main processor detected protocol violations is a hardware failure (since the FPCP cannot return an illegal response primitive).

For FPCP-detected protocol violations, the cause is most likely a software failure that causes a new instruction to be initiated before the previous instruction dialog is completed. In this case, the new instruction dialog is aborted immediately, but the previous instruction dialog may not terminate for some time. (The previous dialog may be completed incorrectly, since the protocol violation is never reported to the previous instruction.)

7.5.4.1 TAKE PRE-INSTRUCTION EXCEPTION. This dialog is utilized by the MC68881 when an arithmetic (OPCLASS 000, 010, or 011) or conditional instruction is initiated and an arithmetic exception is pending from a previous instruction, or when the main processor writes an undefined, reserved command word to the command CIR. In either case, this dialog consists of two write cycles and one read cycle, as shown in Figure 7-27. First, the main processor attempts to initiate a new instruction by writing to the command CIR; it then reads the response CIR to determine the next required action. The MC68881 returns the take pre-instruction exception in this case, indicating the appropriate vector number.

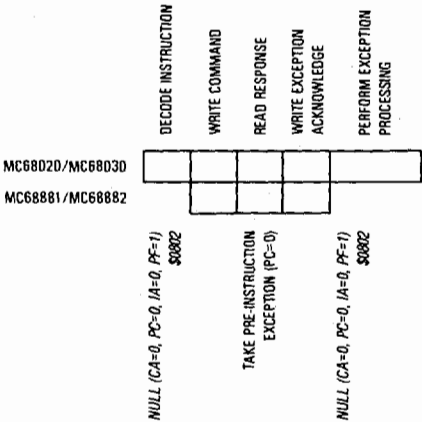


Figure 7-27. Take Pre-Instruction Exception Dialog — MC68881



main processor services this primitive by writing an exception acknowledge to the control CIR and initiating exception processing.

Note that the write of the exception acknowledge causes the response CIR encoding to be changed to the null primitive, thus assuring that the take exception primitive is received by the main processor while avoiding spurious request primitives in non-MPU based systems.

The MC68882 uses a similar dialog for pre-instruction exceptions. However, when the main processor writes an exception acknowledge to the control CIR, the MC68882 does not enter the idle state. Instead, the MC68882 retains the take pre-instruction exception primitive in its response register. Any floating-point instruction other than an FSAVE (or an FRESTORE of the null state) reports the same exception again. An FSAVE (or an FRESTORE of the null state) restores the MC68882 to an idle state, allowing subsequent floating-point instructions to execute. Figure 7-28 shows the dialog which includes the minimum instructions recommended for an exception handler. (Refer to **5.2.2 Exception Handler Code**.)

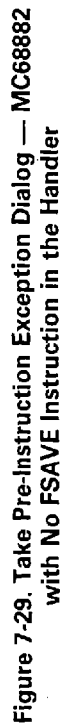
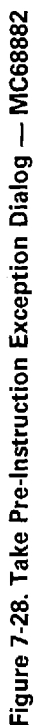
Figure 7-29 shows the dialog that usually occurs when the handler for a pre-instruction exception does not begin with an FSAVE instruction. A protocol violation could occur; in which case, the dialog would not apply. Otherwise, as the dialog shows, at the completion of the exception handler routine, the main processor attempts to execute the same instruction again, and that instruction takes the same exception. If the exception handler included a floating-point instruction but no preceding FSAVE instruction, the floating-point instruction would take the original exception again, nesting the repetitions of the exception handler.

Figure 7-30 shows the dialog that applies when an exception handler does not set the exception pending bit, even though it begins with an FSAVE instruction and closes with an FRESTORE. A protocol violation cannot occur in this case, but because the exception pending bit is not set, the instruction again takes the exception when it is reinitiated after returning from the exception handler.

7.5.4.2 TAKE MID-INSTRUCTION EXCEPTION. The MC68881 uses this dialog only if an exception occurs during the execution of the FMOVE FPM,<ea> instruction. In this case, the protocol for the normal execution of the instruction is followed, and then the mid-instruction exception is reported with the last primitive (in lieu of the null primitive normally used to terminate the dialog). The main processor services this primitive by writing an exception processing acknowledge to the control CIR and initiating exception processing.

The dialog for this operation is shown in Figure 7-31. (For simplicity, this diagram assumes that the destination data format is not packed decimal with a dynamic k factor.) Note that a write of the exception acknowledge causes the response CIR encoding to be changed to the null primitive, thus assuring that the take-exception primitive is received by the main processor while avoiding a spurious request primitive in non-MC68020 or non-MC68030 based systems.

The MC68882 uses the mid-instruction exception if an exception occurs in the FMOVE FPM,<ea> instruction, as the MC68881 does. However, since the MC68882 allows concurrent execution of floating-point instructions, an instruction that has begun execution can report an exception caused by a previous instruction. When the previous instruction



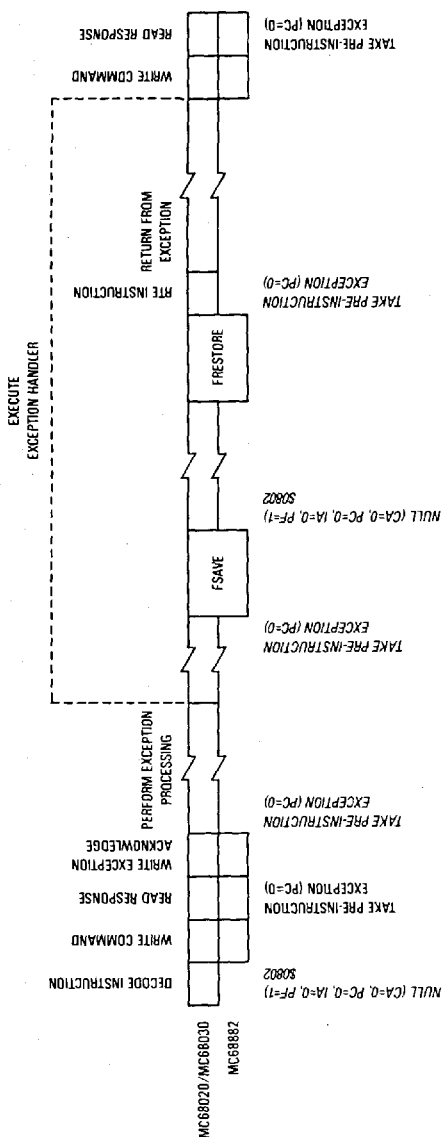


Figure 7-30. Take Pre-Instruction Exception Dialog — MC68882 with No BSET Instruction in the Exception Handler

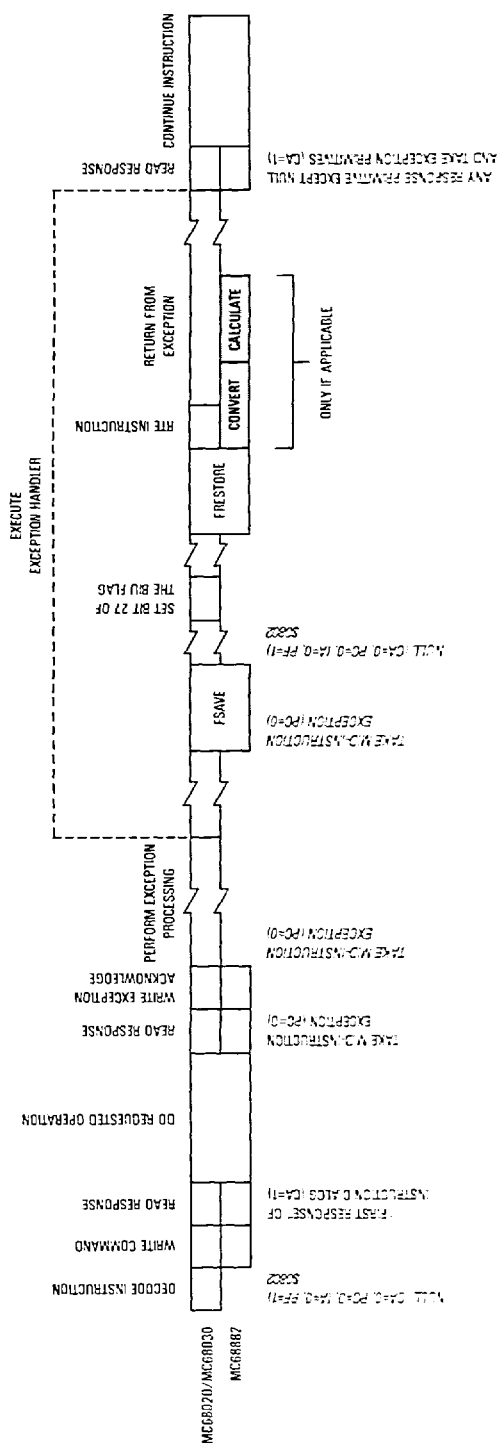


Figure 7-31. Take Mid-Instruction Exception Dialog — MC68881

is an exception pending, the exception is reported on the next read operation of the response CIR. Therefore, all reads of the response CIR must be ready to take an exception. If the read of the response CIR occurs in the middle of an instruction, a take mid-instruction exception is taken. The dialog shown in Figure 7-32 is a generic case that applies to all instructions and to every read of the response CIR after the instruction has issued its first response. The first response is the response issued by the conversion unit (CU) or the arithmetic processing unit (APU) to the bus interface unit (BIU), not the null primitive (CA=1, IA=1) at the beginning of an instruction when (in an MC68881) the APU is busy or (in the MC68882) the CU is busy.

The MC68882 dialog is similar, except that the exception handler must begin with an FSAVE instruction. The significant difference is that the write exception acknowledge operation does not cause the MC68882 to return a null primitive. The FSAVE instruction of the exception handler changes the primitive to a null primitive.

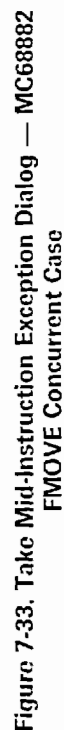
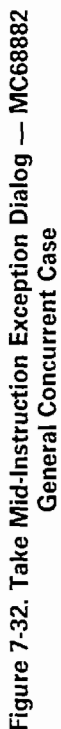
In the MC68882, an instruction that is executing in the arithmetic processing unit (APU) concurrently with another instruction in the conversion unit (CU) may cause an exception. The instruction in the CU reports the exception as a mid-instruction exception when it completes. Figure 7-32 shows the dialog for this case using a general instruction dialog. When the previous instruction causes an exception, the read response CIR operation for the current instruction reads a take mid-instruction primitive, and the main processor performs exception processing. As in the nonconcurrent case, the write exception acknowledge operation does not cause the MC68882 to return a null primitive. The FSAVE instruction of the exception handler changes the primitive to a null primitive. Figure 7-33 shows the same case but with a specific instruction (FMOVE FPM,<ea>) in the CU.

When the exception handler does not contain an FSAVE instruction, the take mid-instruction exception primitive is not replaced by a null primitive, and the next floating-point instruction takes the exception again. Figure 7-34 shows the dialog for a mid-instruction exception that uses an incomplete handler.

Figure 7-35 shows the concurrent case using an exception handler that does not include the BSET instruction. Following the return from the exception handler, the main processor reads the take mid-instruction exception primitive from the response CIR and performs exception processing again for the same exception.

7.5.4.3 MID-INSTRUCTION INTERRUPT. This dialog is utilized by the FPCP only if an interrupt is pending during the calculation phase of the FMOVE FPM,<ea> instruction. In this case, the protocol for the normal execution of the instruction is followed except that the main processor performs exception processing for the interrupt, executes the interrupt handler, and returns to the point where the dialog was suspended in the middle of the execution of the instruction by the FPCP. From the perspective of the FPCP, the dialog appears to follow the normal sequence, with a long delay between successive reads of the response CIR by the main processor.

The dialog for this operation is shown in Figure 7-36. (For simplicity, this diagram assumes that the destination data format is not packed decimal with a dynamic k factor.) Note that it is possible (even probable) that the conversion of the output operand is completed before the main processor returns from the interrupt. Thus, the response CIR is prepared to return the evaluate effective address and transfer data primitive as soon as the main processor returns. Since the read of this primitive causes the FPCP to discard it and change the



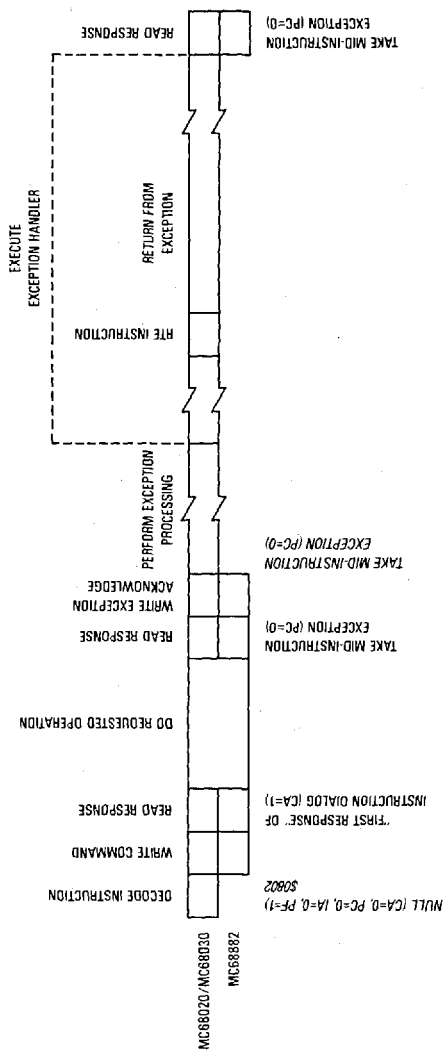


Figure 7-34. Take Mid-Instruction Exception Dialog — MC68882 with No FSAVE Instruction in the Handler

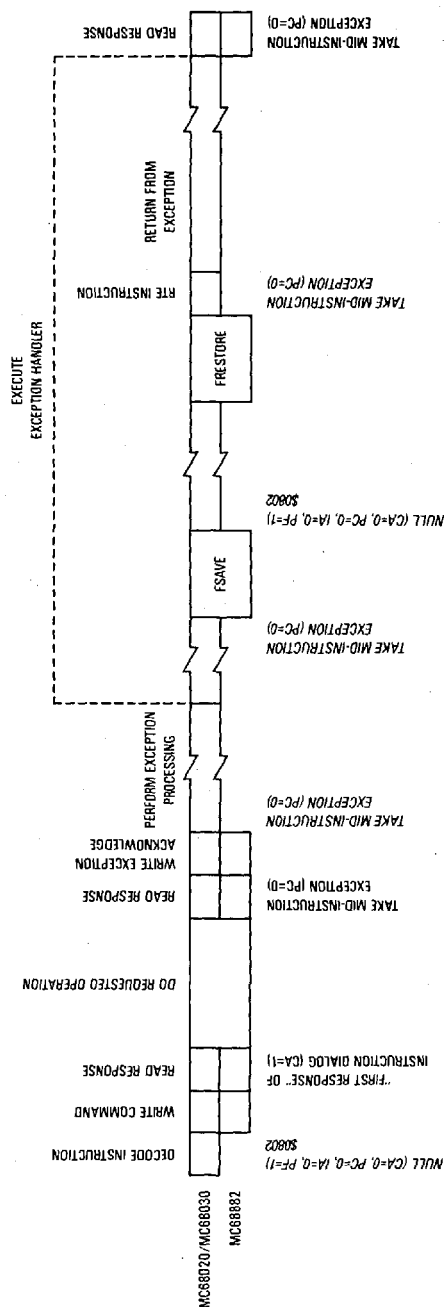


Figure 7-35. Take Mid-Instruction Exception Dialog — MC68882 with No BSET Instruction in the Exception Handler

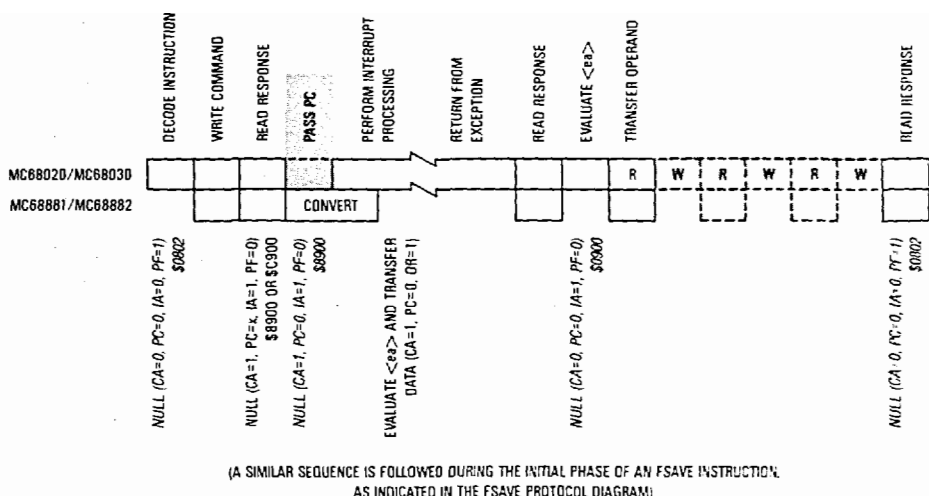


Figure 7-36. Mid-Instruction Interrupt Dialog

7

response CIR encoding to the null primitive, the interrupt handler (or any other routine) must not casually read the response CIR to determine the status of the FPCP, or the suspended protocol is disrupted. Rather, the only valid method for checking the status of the FPCP is to execute the FSAVE instruction and examine the state frame that is generated; followed by an FRESTORE instruction to reinstate the previous context of the FPCP.

7.5.4.4 TAKE BSUN EXCEPTION. This dialog is utilized by the FPCP when a conditional instruction is initiated by writing one of the IEEE nonaware conditional predicates to the condition CIR with the SNAN enable bit and the NAN condition code bit set. The dialog is shown in Figure 7-37.

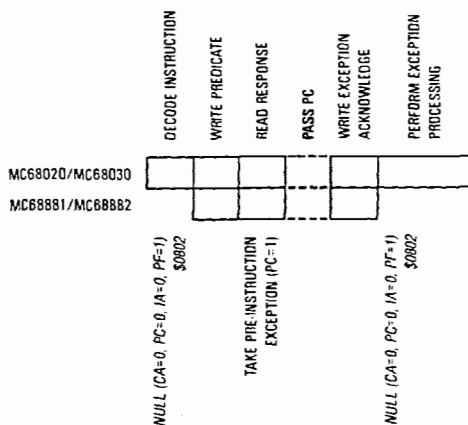


Figure 7-37. Take BSUN Exception Dialog

the main processor reads the response CIR to receive the true/false result of the conditional evaluation, the FPCP returns the take pre-instruction exception primitive instead of the null primitive. In order to update the FPIAR, the PC bit of this primitive is also set. The main processor services this primitive by transferring the program counter, writing an exception acknowledge to the control CIR, and then initiating exception processing. Although the MC68020 or MC68030 always performs the program counter transfer when it is requested, other main processors may choose to ignore this request from an MC68881 without incurring a protocol violation. The MC68882 returns a protocol violation whenever the main processor ignores a request for transfer of the program counter.

Note that the write of the exception acknowledge causes the response CIR encoding to be changed to the null primitive, thus assuring that the take exception primitive is received by the main processor while avoiding spurious request primitives in non-MC68020 or non-MC68030 based systems.

7.5.4.5 F-LINE EMULATOR EXCEPTION. This dialog is utilized by the FPCP when a general instruction is initiated, and the value written to the command CIR is not a legal FPCP command word encoding. In this case, the dialog consists of two write cycles and one read cycle, as shown in Figure 7-38. First, the main processor attempts to initiate a new instruction by writing to the command CIR; it then reads the response CIR to determine the appropriate action to be taken. In this case, the first read of the response CIR returns a take exception primitive with the F-line emulator vector number. The main processor services this primitive by writing an exception acknowledge to the control CIR and initiating exception processing.

Note that the write of the exception acknowledge causes the response CIR encoding to be changed to the null primitive while avoiding spurious request primitives in non-MC68020 or non-MC68030 based systems.

7.5.4.6 FORMAT EXCEPTION, FSAVE INSTRUCTION. This dialog is utilized by the FPCP when an FSAVE or FRESTORE instruction dialog is interrupted by an attempt to initiate a

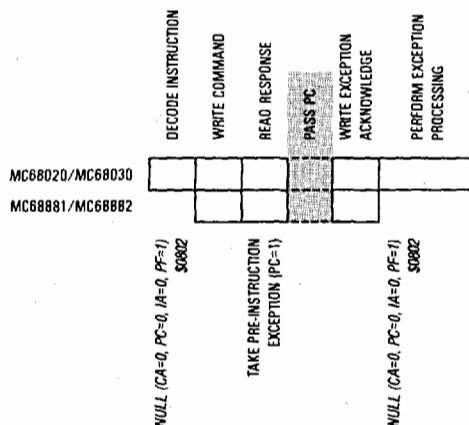


Figure 7-38. Take F-Line Emulator Exception Dialog

an FSAVE instruction (by reading from the save CIR). In this case, the FPCP returns the valid format word to signal the illegal nesting of the FSAVE instruction. The main processor services this format word by writing an abort to the control CIR and initiating exception processing. The dialog for this operation is shown in Figure 7-39.

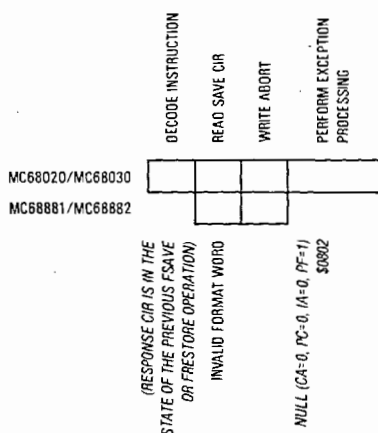


Figure 7-39. FSAVE Format Exception Dialog

Since the MPU writes an abort to the FPCP in response to the illegal format word, the FSAVE or FRESTORE that was interrupted by the nested FSAVE is destructively aborted with no indication to the suspended instruction of this occurrence. Thus, a suspended save operation continues to read the "frame" from the operand CIR if it is resumed, even though the data in the operand CIR is not valid. Likewise, a suspended restore operation writes the remainder of the frame to the operand CIR if it is resumed, even though the data written is ignored and the restore operation is not performed. Due to the destructive behavior of a nested FSAVE instruction, programmers must be certain that the FPCP is not executing an FSAVE or FRESTORE instruction prior to an attempt to execute a new FSAVE instruction. If there is a possibility that a nested FSAVE might occur, the MPU MOVES instruction might be used to read the save CIR before the FSAVE is executed. If the value returned from the save CIR is the illegal format word, then the new FSAVE should be postponed. Reading the save CIR in this manner is not destructive.

7.5.4.7 FORMAT EXCEPTION, FRESTORE INSTRUCTION. This dialog is utilized by the FPCP when an FRESTORE instruction is initiated by writing an invalid format word value to the restore CIR. (In this context, the term invalid format value refers to any value that is not a null, idle, or busy format word value recognized by the FPCP.) In this case, the FPCP returns the explicit invalid format word (\$02xx) to signal the unrecognized format word value. The main processor services this format word by writing an abort to the control CIR and initiating exception processing. The dialog for this operation is shown in Figure 7-40. Note that this is a destructive exception since any instruction that was executing is aborted when the FRESTORE instruction is initiated. However, this should not be detrimental since a successful restore operation also aborts any previously executing instruction.

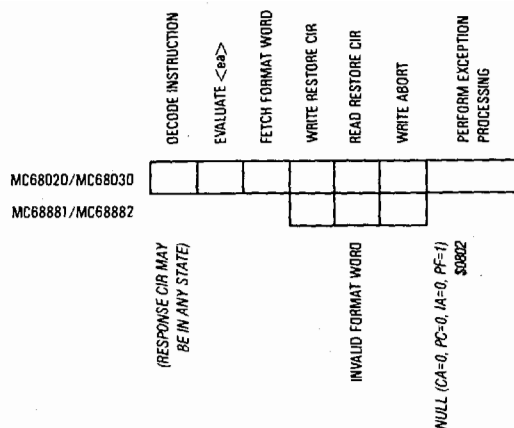


Figure 7-40. FRESTORE Format Exception Dialog

